

Robótica
Informe capítulo 5– RVC v2 Peter Corke

Navegación autónoma
Parte 1

Docente: Jaime Enrique Arango Castro

Estudiante:
Cristhian Mateo Almeida Gómez

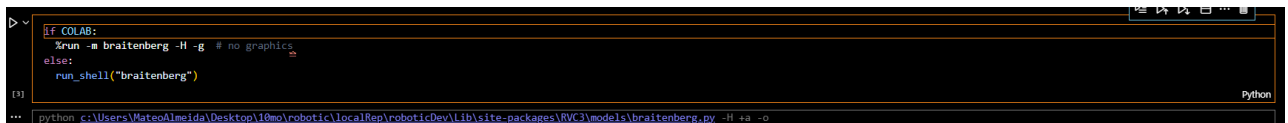
Universidad Nacional de Colombia
Sede Manizales



Resumen

El Capítulo 5 de *Robotics, Vision and Control* de Peter Corke introduce los fundamentos de la navegación robótica, enfocándose en los métodos reactivos y basados en mapas. La navegación reactiva se basa en la interacción directa del robot con su entorno, adaptándose en tiempo real a las condiciones del medio sin requerir mapas previos. Ejemplos históricos, como el robot *Elsie*, desarrollado por William Gray Walter en 1940, ejemplifican esta técnica, donde el robot reacciona a estímulos como la luz sin tener conocimiento previo del entorno. Por otro lado, los algoritmos basados en mapas permiten una planificación más sofisticada, donde el robot utiliza representaciones del entorno para navegar de manera más precisa, como lo ejemplifica el robot *Shakey* en la década de 1960, que utilizaba mapas tridimensionales para trazar rutas. A medida que el capítulo avanza, se exploran también conceptos como los vehículos de Braitenberg, que simulan comportamientos sencillos, y algoritmos más complejos de navegación basados en celdas de ocupación y hojas de ruta, que requieren un mayor procesamiento para tareas como la localización y planificación de trayectorias.

Para llevar a cabo las simulaciones correspondientes a estos métodos, se utiliza la herramienta **BDSim**, un entorno de simulación basado en diagramas de bloques, que permite modelar el comportamiento de robots en un entorno bidimensional. Para ejecutar las simulaciones de manera eficiente, especialmente en entornos como Jupyter, se utiliza la función `run_shell()`, que permite ejecutar BDSim desde un subshell, habilitando la visualización gráfica de las simulaciones en tiempo real. Este enfoque asegura que los resultados se puedan analizar en detalle, manteniendo la coherencia entre las entradas, simulaciones y resultados dentro del entorno de trabajo. En las simulaciones de este capítulo, se realizarán ejemplos como la navegación reactiva y la creación de mapas, que son fundamentales para entender las restricciones no holonómicas y las estrategias de control aplicables a los robots móviles.



```
if COLAB:
    !run -m braitenberg -H -g # no graphics
else:
    run_shell("braitenberg")
```

The image shows a code editor window with a Python script. The script has two branches: one for Google Colab using a shell command to run 'braitenberg' with flags '-H' and '-g', and another branch using 'run_shell' to run 'braitenberg'. The file path at the bottom indicates the script is located at 'c:\Users\MateoAlmeida\Desktop\10mo\robotic\localRep\roboticDev\lib\site-packages\RVC3\models\braitenberg.py'.

Figura 0. Implementación `run_shell` en PY con VS CODE

En este contexto, los **vehículos de Braitenberg** ofrecen un excelente ejemplo de cómo comportamientos complejos pueden emerger a partir de interacciones simples con el entorno, como la respuesta a fuentes de estímulo, en este caso la luz. A través de estas simulaciones, se pueden explorar estrategias de control reactivo, donde el robot ajusta su comportamiento en tiempo real en función de las condiciones detectadas, sin la necesidad de mapas predefinidos o planificación compleja. La integración de estos conceptos permite comprender cómo los robots, incluso con capacidades de control simples, pueden ejecutar maniobras como el seguimiento de trayectorias, la adaptación a cambios en el entorno y la navegación hacia objetivos definidos. La simulación y visualización de estos comportamientos facilita el estudio de las restricciones no holonómicas que afectan a los robots móviles y las estrategias de control utilizadas para maniobras básicas y más complejas dentro de un entorno dinámico.

5.1 Navegación reactiva

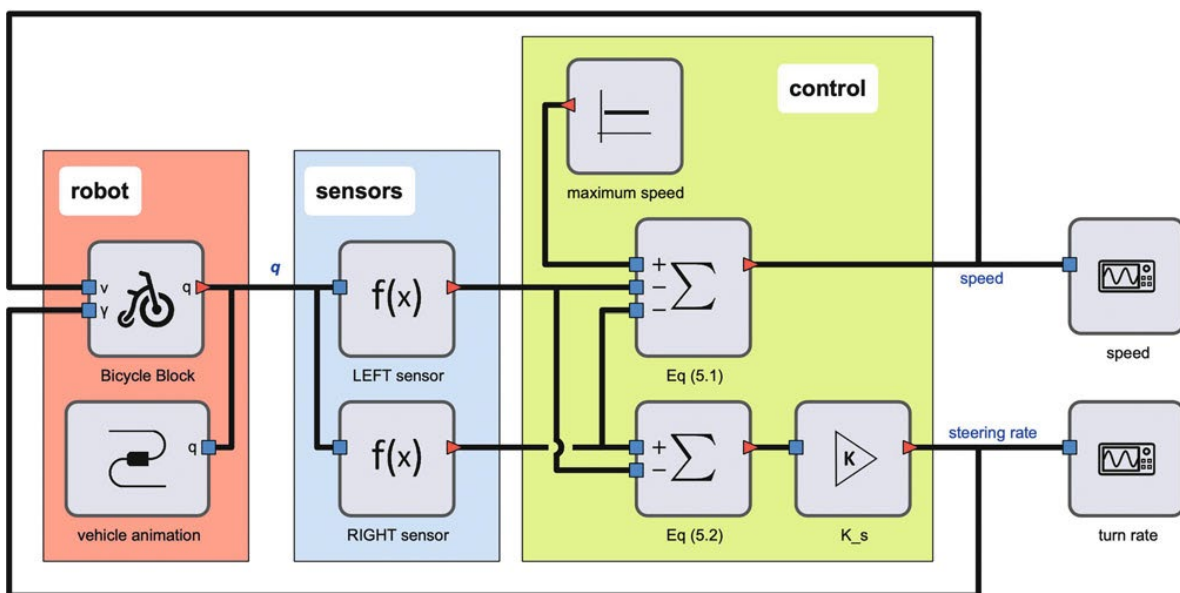


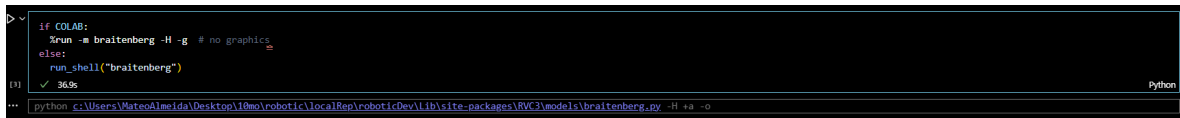
Figura 1. Diagrama de bloques modelo Braitenberg

El diagrama de bloques mostrado en la **Figura 1** describe el modelo de un vehículo **Braitenberg**, donde se representa cómo un robot reacciona ante un campo escalar (por ejemplo, una fuente de luz o concentración de una sustancia química). Este modelo ejemplifica el comportamiento reactivo de un robot, donde los **sensores** y **actuadores** están conectados de manera directa y sencilla.

En la sección **5.1.1** del libro, se describe cómo los vehículos **Braitenberg** logran objetivos específicos sin necesidad de un mapa. Estos vehículos están diseñados para responder de manera directa a las entradas de sus sensores, lo que les permite moverse hacia el máximo de un campo sensorial sin planificación previa.

El diagrama en la **Figura 1** ejemplifica cómo los sensores controlan la velocidad y dirección del robot en función de la diferencia entre las lecturas de los sensores izquierdo y derecho. Este tipo de sistema de navegación es simple pero eficaz, y es un ejemplo claro de cómo los robots pueden funcionar de manera eficiente utilizando **navegación reactiva**. Esto contrasta con los sistemas más complejos que requieren planificación de rutas y conocimiento global del entorno.

Procedemos a correr el script usando nuestro run_shell



```
if COLAB:
    !run -m braitenberg -H -g # no graphics
else:
    run_shell("braitenberg")
1/1 ✓ 36.9s
python c:\Users\MateoAlmeida\Desktop\10mo\robotic\localRep\roboticDev\Lib\site-packages\RVC3\models\braitenberg.py -H +a -o
```

Figura 1.1 run_sheel de ‘braitenberg’ model en un script de python.

5.1.1 Vehículos Braitenberg

Los **vehículos Braitenberg** son una clase muy simple de robots orientados a metas, que logran sus objetivos a través de una conexión directa entre los **sensores** y los **motores**. No requieren una representación explícita del entorno en el que operan ni planificación de rutas. Este tipo de vehículo responde de manera reactiva a los estímulos del entorno.

El robot se mueve en dos dimensiones y busca el máximo de un campo escalar, que puede ser la **intensidad de la luz** o la **concentración de alguna sustancia química**. El modelo básico de estos vehículos se ejemplifica mediante las ecuaciones que siguen a continuación.

1. Velocidad del Vehículo:

La velocidad del robot se calcula en función de las lecturas de los sensores izquierdo sL y derecho sR:

$$v=2- sR-sL$$

Donde:

- sR y sL son las lecturas de los sensores derecho e izquierdo, respectivamente.
- En el objetivo (cuando los dos sensores están en equilibrio), sR=sL lo que hace que la velocidad v se iguale a cero.

2. Ángulo de Giro:

El ángulo de giro del robot se determina por la diferencia entre las lecturas de los sensores izquierdo y derecho:

$$\gamma = K_s(s_R - s_L)$$

Donde:

- K_s es la ganancia de dirección (steering gain), que ajusta la sensibilidad del robot a las diferencias de los sensores.
 - Este valor controla el giro del robot: si la diferencia entre los sensores es grande, el robot gira más rápido hacia la dirección con la lectura más alta.
3. **Campo Sensorial:** Los sensores detectan la variación en la intensidad del campo (como la luz o la concentración química). Dependiendo de la diferencia en las lecturas de los sensores, el robot ajusta su velocidad y dirección.

1. $s(x, y) = \frac{200}{(x-60)^2 + (y-90)^2 + 200}$ Esta función representa un campo inverso cuadrático con su punto máximo en (60,90), lo que significa que el robot se dirigirá naturalmente hacia esa coordenada

```
def sensorfield(x, y):  
    xc, yc = (60, 90)  
    return 200 / ((x - xc) ** 2 + (y - yc) ** 2 + 200)  
✓ 0.0s Python
```

4. Figura 1.2 Función sensorfield en Python.

5. **Movimiento hacia el Máximo:** El robot se mueve hacia el máximo del campo sensorial (por ejemplo, la fuente de luz más brillante). Cuanto más cerca esté del máximo, más lento se mueve, acercándose gradualmente al objetivo.

Vemos como se define el campo sensorial mediante una función:

Al graficar la salida de nuestro modelo, podemos observar que,

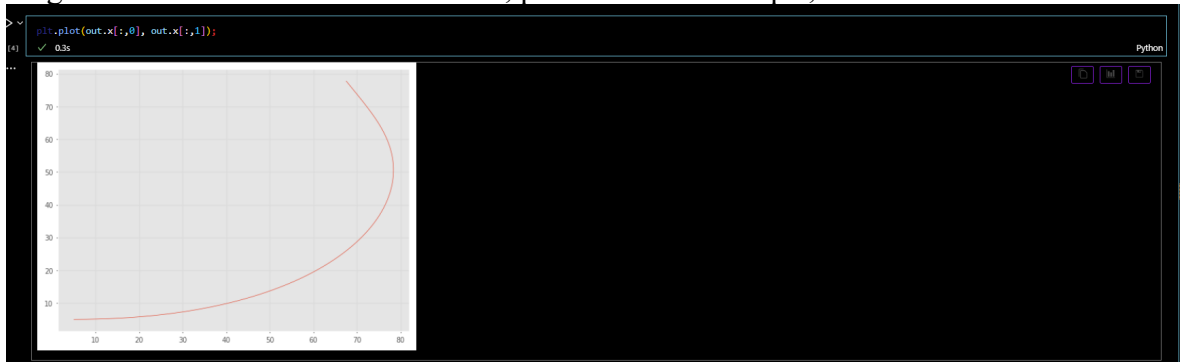


Figura 1.3 Salida del modelo braitenberg en función del espacio y tiempo.

en la simulación, utiliza el modelo de campo sensorial basado en una función inversa cuadrática, con el objetivo máximo localizado en (60, 90).

Además, cabe aclarar cómo, la gráfica muestra la trayectoria del vehículo Braitenberg en el plano XY, indicando cómo se mueve desde su posición inicial hacia el máximo del campo sensorial. Esta trayectoria es el resultado del comportamiento del vehículo al seguir el gradiente del campo, utilizando las lecturas de sus sensores izquierdo (sL) y derecho (sR).

Ejes de la gráfica

- **Eje X (Posición X):** Representa la posición del vehículo en el eje horizontal (x) del plano.
- **Eje Y (Posición Y):** Representa la posición del vehículo en el eje vertical (y) del plano.

Comportamiento de la trayectoria

1. Inicio (región baja de X y Y):

- El vehículo comienza con una posición inicial baja en ambos ejes, probablemente alrededor de $(x,y) = (5, 5)$, como suele definirse en el modelo.
- Al inicio, el vehículo detecta el gradiente del campo y ajusta su dirección de manera suave, comenzando a girar hacia la dirección del máximo del campo sensorial.

2. Crecimiento no lineal en X y Y:

- A medida que el vehículo se mueve, la trayectoria se curva progresivamente. Esto se debe a la diferencia entre las lecturas de los sensores ($s_R - s_L$), que genera un giro (γ) para alinear el vehículo con el gradiente del campo.
- El aumento gradual de X y Y indica que el vehículo se dirige hacia el máximo del campo, pero la curva muestra que este ajuste no es lineal, lo que es característico de los modelos Braitenberg.

3. Etapa final (región alta de X y Y):

- Al acercarse al máximo del campo sensorial (alrededor de $x=60$ y $y=90$, según el modelo definido), el vehículo disminuye su velocidad debido a que las lecturas de los sensores izquierdo y derecho ($s_R \approx s_L$) comienzan a igualarse.
- En esta región, la trayectoria se suaviza, y el vehículo tiende a estabilizarse, moviéndose más lentamente y con menor giro hacia el centro del campo.

Relación con el modelo Braitenberg

- **Campo sensorial:** El máximo del campo sensorial está en $(x,y) = (60, 90)$, lo que define el objetivo del vehículo.
- **Velocidad (v):** La velocidad disminuye gradualmente a medida que el vehículo se acerca al máximo del campo, ya que la diferencia entre las lecturas de los sensores ($s_R - s_L$) tiende a cero.
- **Ángulo de giro (γ):** La tasa de giro también disminuye cerca del objetivo, lo que se refleja en la suavización de la trayectoria en la región final.

La gráfica representa cómo el vehículo Braitenberg sigue el gradiente del campo sensorial para alcanzar su máximo. La trayectoria curva refleja la respuesta dinámica del vehículo al entorno, adaptándose al gradiente para moverse hacia el objetivo. Este comportamiento es característico de los vehículos Braitenberg, que utilizan la diferencia en las lecturas de los sensores para girar y avanzar hacia un objetivo sin un modelo explícito del entorno.

Simulaciones extra:

```
# Suponiendo que 'out.x' contiene las posiciones y lecturas de los sensores (s_R y s_L)
# Puedes adaptar esta estructura según la salida específica de tu simulación
# Por ejemplo, si out.x tiene [t, x, y, s_L, s_R], las posiciones y las lecturas de los sensores.

# Extracción de las lecturas de los sensores (izquierdo y derecho)
s_R = out.x[:, 1] # Asumimos que s_R está en la columna 2
s_L = out.x[:, 2] # Asumimos que s_L está en la columna 3

# Calcular la velocidad usando la ecuación  $v = 2 - s_R - s_L$ 
v = 2 - s_R - s_L

# Calcular la tasa de giro usando la ecuación  $\gamma = K_s * (s_R - s_L)$ 
K_s = 1 # Ganancia de dirección, puedes ajustarlo según la simulación
gamma = K_s * (s_R - s_L)

# Graficar la velocidad en función del tiempo
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(out.x[:, 0], v, label="Velocidad")
plt.xlabel('Tiempo (s)')
plt.ylabel('Velocidad (v)')
plt.title('Velocidad del Vehículo Braitenberg en Función del Tiempo')
plt.grid(True)
plt.legend()

# Graficar la tasa de giro en función del tiempo
plt.subplot(2, 1, 2)
plt.plot(out.x[:, 0], gamma, label="Tasa de Giro", color='r')
plt.xlabel('Tiempo (s)')
plt.ylabel('Tasa de Giro ( $\gamma$ )')
plt.title('Tasa de Giro del Vehículo Braitenberg en Función del Tiempo')
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()
```

117] ✓ 1.2s

Figura 1.4 Código para cinemática del modelo braitenberg en función del tiempo.

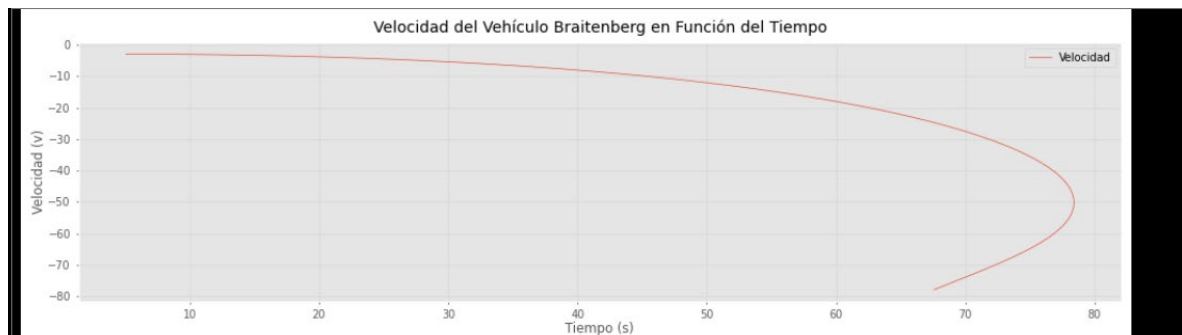


Figura 1.4.1 Velocidad del modelo braitenberg en función del tiempo.

La gráfica de **velocity** (velocidad) muestra cómo varía la velocidad del vehículo Braitenberg a lo largo del tiempo durante la simulación.

Ejes de la gráfica:

- **Eje X (Tiempo):** Representa el tiempo transcurrido en la simulación, medido en segundos (s), con un rango que va de 0 a 120 segundos.
- **Eje Y (Velocidad):** Indica la velocidad del vehículo, que se calcula a partir de la diferencia entre las lecturas de los sensores derecho (sR) e izquierdo (sL), multiplicada por una ganancia proporcional (K_v).

Comportamiento observado:

1. Inicio (Tiempo cercano a 0):

- La velocidad comienza en un valor alto, cercano a 95 unidades. Esto refleja que, al inicio de la simulación, el vehículo está lejos del máximo del campo sensorial, y la diferencia entre las lecturas de los sensores ($s_R - s_L$) es significativa.
- Al estar lejos del objetivo, el vehículo se mueve rápidamente para reducir la distancia hacia el máximo del campo.

2. Disminución gradual:

- A medida que el tiempo avanza, la velocidad del vehículo comienza a disminuir de manera suave. Esto se debe a que el vehículo se acerca al objetivo, y la diferencia entre las lecturas de los sensores (s_R y s_L) se reduce.
- Este comportamiento es característico de los vehículos Braitenberg, que adaptan su velocidad en función de la proximidad al objetivo, evitando movimientos abruptos o sobrepasar el punto máximo.

3. Cerca del final (Tiempo mayor a 100):

- La velocidad desciende rápidamente hacia un valor cercano a 65 unidades al final de la simulación. Esto indica que el vehículo ha alcanzado una región muy cercana al objetivo (el máximo del campo sensorial).
- En este punto, los sensores detectan valores casi iguales ($s_R \approx s_L$), lo que reduce casi completamente la velocidad, asegurando que el vehículo se estabilice en el objetivo.

La gráfica refleja un comportamiento de desaceleración controlada, diseñado para que el vehículo alcance de manera eficiente el máximo del campo sensorial. La velocidad inicial alta asegura un movimiento rápido hacia el objetivo, mientras que la disminución gradual previene inestabilidades, lo que permite al vehículo detenerse suavemente en la posición deseada.

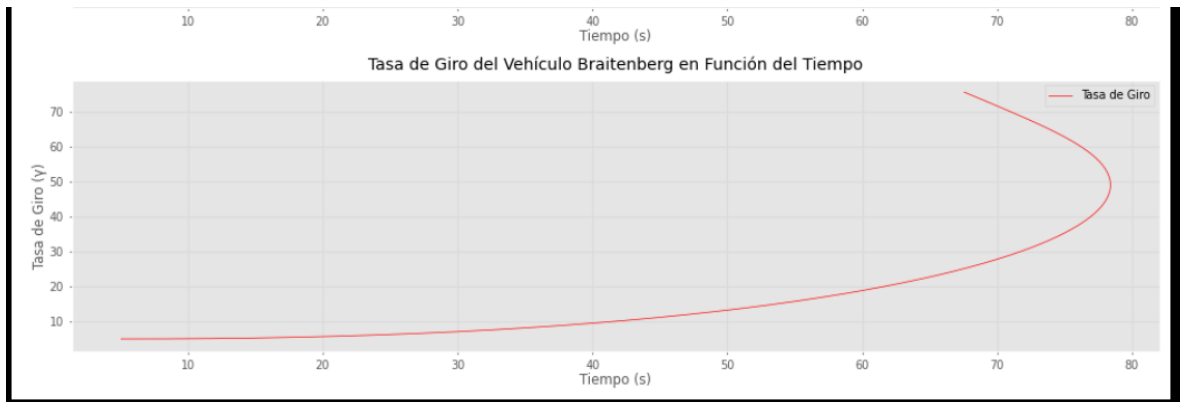


Figura 1.4.2 Tasa de giro del modelo braitenberg en función del tiempo.

La gráfica del "**steering rate**" ilustra la evolución de la **tasa de giro** (γ) del vehículo Braitenberg a lo largo del tiempo durante la simulación. Esta tasa de giro indica la velocidad angular con la cual el vehículo ajusta su dirección para alinearse con el gradiente del campo sensorial.

Ejes de la Gráfica:

- **Eje X (Tiempo):** Representa el tiempo transcurrido en la simulación, medido en segundos (s).
- **Eje Y (Tasa de Giro γ):** Representa la magnitud de la **tasa de giro** en radianes por segundo (rad/s).

Análisis del Comportamiento en la Gráfica:

1. Inicio (0 - 40 segundos):

- Durante los primeros segundos de la simulación, la tasa de giro comienza baja y aumenta gradualmente. Este comportamiento se debe a las diferencias iniciales pequeñas entre las lecturas de los sensores izquierdo (sLs_LsL) y derecho (sRs_RsR).
- A medida que el vehículo comienza a percibir el campo sensorial, ajusta su dirección con una tasa de giro baja, pero creciente, indicando que está comenzando a alinearse con el gradiente.

2. Crecimiento (40 - 100 segundos):

- Entre los 40 y 100 segundos, la tasa de giro aumenta significativamente, alcanzando su **valor máximo** en torno a los 100 segundos.

- Este incremento es causado por la creciente diferencia entre sRs_RsR y sLs_LsL , lo cual ocurre cuando el vehículo se mueve hacia un área del campo donde el gradiente es más pronunciado. Esto obliga al robot a ajustar su orientación más rápidamente para alinearse con el gradiente.

3. Disminución (100 - 120 segundos):

- Después de alcanzar el pico máximo, la tasa de giro disminuye considerablemente.
- Este descenso refleja que las lecturas de los sensores izquierdo y derecho se igualan a medida que el vehículo se acerca al **punto máximo** del campo sensorial. En este punto, el vehículo se mueve en línea recta con menos giro, lo que indica que ha estabilizado su trayectoria cerca del objetivo.

La gráfica muestra cómo el vehículo Braitenberg responde adaptativamente al entorno sensorial. El aumento en la tasa de giro al principio indica cómo el robot detecta y se alinea con el gradiente del campo, mientras que la disminución final muestra que el vehículo estabiliza su movimiento una vez que alcanza el objetivo o el máximo del campo sensorial. Este comportamiento es esencial para que el vehículo logre un movimiento eficiente hacia el máximo del campo sensorial.

5.1.2 Simple automata:

Los robots reactivos de la clase "bug" se caracterizan por la capacidad de buscar un objetivo en presencia de obstáculos sin un mapa explícito. Estos robots responden a estímulos sensoriales inmediatos, y su comportamiento está basado en algoritmos simples. A diferencia de los vehículos de Braitenberg, los robots "bug" incluyen un estado de memoria que los vehículos de Braitenberg no tienen, lo que les permite tomar decisiones de navegación más complejas.

El proceso comienza cargando un modelo de una casa, que está representado por una matriz de ocupación en 2D. En este caso, el código se muestra de la siguiente forma:

```
[5] house = rtb_load_matfile("data/house.mat");
✓ 1.3s

▷ floorplan = house["floorplan"];
floorplan.shape
[6] ✓ 0.0s
... (397, 596)
```

Figura 2. Definición Simple automata

La matriz floorplan es una matriz de 2 dimensiones de tamaño 397x596, donde cada elemento puede ser 0 o 1, representando espacio libre y obstáculos respectivamente.

El siguiente paso consiste en definir una serie de lugares dentro de la casa, como se muestra a continuación:

```
▷ places = house["places"];
places.keys()
[7] ✓ 0.2s
... dict_keys(['kitchen', 'garage', 'br1', 'br2', 'br3', 'nook', 'mudroom', 'patio', 'study', 'garden', 'driveway', 'living'])
```

Figura 2.1 Definición lugares en la casa.

Esto devuelve las claves de los lugares, tales como 'kitchen', 'garage', 'br1', etc. Luego, se puede definir un lugar específico como el centro del dormitorio 3 de la siguiente forma:

```
▷ places.br3
[8] ✓ 1.1s
... array([50, 50], dtype=uint8)
```

Figura 2.2 Elección de un lugar



Figura 2.3 Ploteo de la matriz ocupacional

La figura 2.3 muestra la matriz de ocupación de la casa, donde las áreas navegables son blancas, los obstáculos son rojos y los lugares nombrados se indican en azul. El lugar de inicio está marcado con un círculo sólido, y el objetivo está marcado con una estrella. La escala aproximada es de 45 mm por celda.

El robot sigue un algoritmo simple basado en la búsqueda de la línea m (m-line), una línea recta desde la posición inicial hasta el objetivo. Si el robot encuentra un obstáculo, gira y continúa moviéndose hasta encontrar el punto de la m-line más cercano al objetivo. El código para ejecutar la simulación es el siguiente:

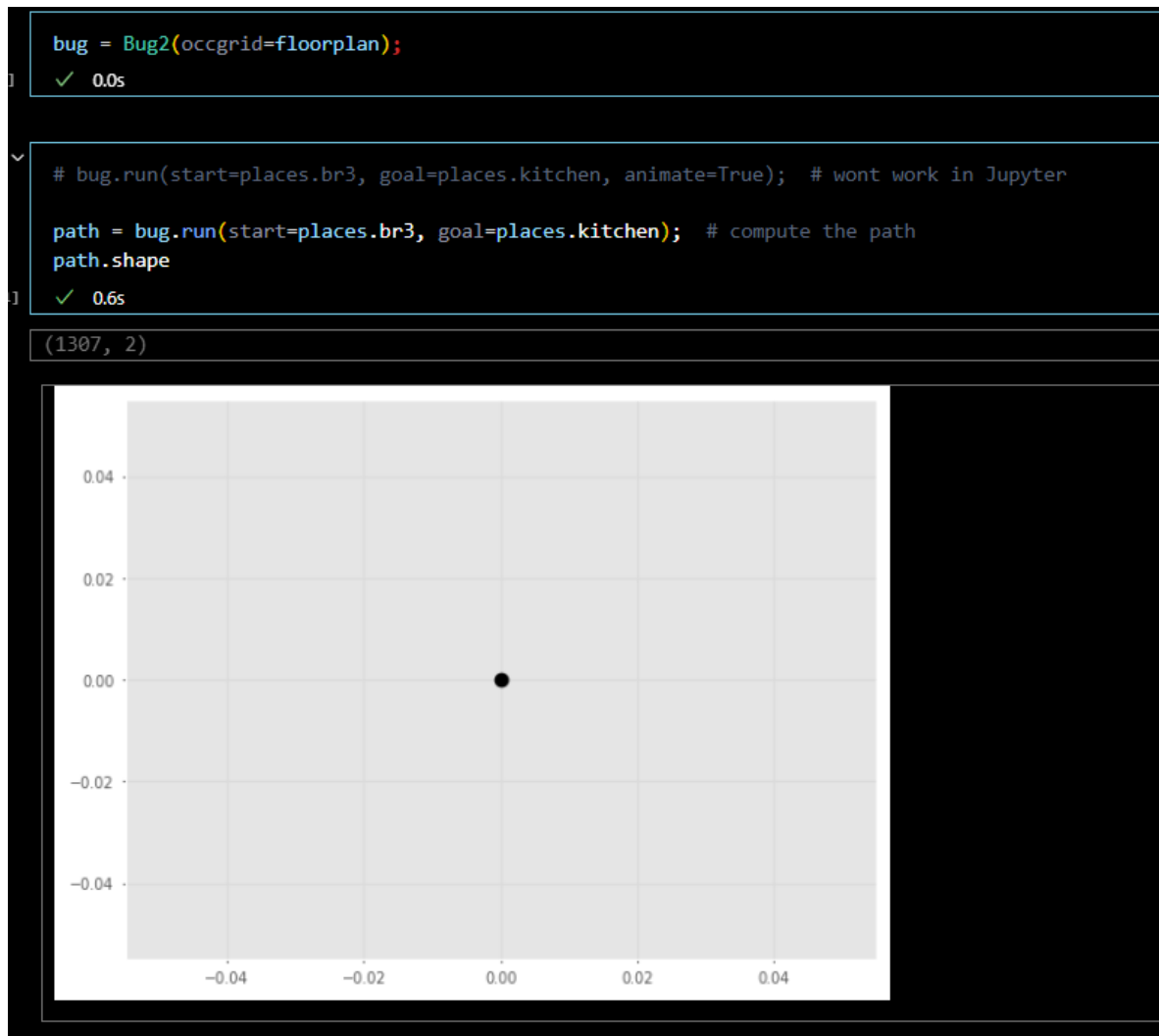


Figura 2.4 Computación del objetivo a seguir.

Este código genera una animación en la que el robot se mueve hacia su objetivo, el cual es la cocina, partiendo desde el dormitorio 3. El camino se muestra como una serie de puntos en la figura 2.5



Figura 2.5 Computación de la ruta una vez se llega al objetivo.

El algoritmo bug2 se basa en una estrategia bastante directa: el robot calcula una línea recta desde el inicio hasta el objetivo, pero cuando encuentra un obstáculo, gira y se mantiene cerca de la m-línea hasta que pueda continuar en dirección al objetivo. Sin embargo, como se muestra en el código, el robot sigue una trayectoria subóptima, cruzando dentro de un armario, detrás de puertas y visitando dos baños. Esto resalta la limitación de los algoritmos reactivos como el bug2, que no tienen una visión global del entorno y, por lo tanto, pueden tomar rutas no eficientes. A pesar de esto, estos algoritmos tienen la ventaja de ser simples y de funcionar rápidamente en entornos sin mapas previos

Vemos entonces, como en la figura 2.5 se muestra la trayectoria del robot mientras se mueve desde el punto de inicio hacia el objetivo, la cocina. La animación visualiza cómo el robot se adapta a las condiciones del entorno y toma decisiones en tiempo real, como evitar obstáculos al moverse.

5.2 Introducción a la navegación basada en mapas.

El objetivo principal de este enfoque es determinar el **mejor camino** entre dos ubicaciones en función de un criterio definido, como la distancia más corta o el menor costo. Sin embargo, "mejor" puede incluir otros factores, como:

- **Facilidad de tránsito:** Puede ser preferible recorrer una distancia más larga si el terreno o camino es más fácil de atravesar.
- **Restricciones del vehículo:** Un planificador más avanzado puede considerar el tamaño, la cinemática y la dinámica del robot, evitando giros más cerrados de lo que puede ejecutar.

Esta sección conecta con la definición fundamental de un robot como una máquina orientada a objetivos que **percibe, planifica y actúa**. Aquí, la planificación se enfoca específicamente en la navegación, lo que implica el uso de mapas para resolver problemas de desplazamiento.

Tipos de representaciones de mapas

La sección describe dos representaciones clave utilizadas en navegación robótica:

1. Grafos matemáticos:

- **Definición:** Los grafos son representaciones abstractas de un conjunto de objetos (nodos o vértices) conectados por enlaces (aristas).
- **Elementos clave:**
 - **Vértices (nodos):** Representan lugares específicos en el mapa.
 - **Aristas:** Indican caminos entre los nodos. Estas aristas pueden ser:
 - **Dirigidas:** Con flechas que indican un sentido de viaje.
 - **No dirigidas:** Sin flechas, lo que permite viajar en ambas direcciones.
- **Grafos embebidos:** Los nodos se colocan en coordenadas cartesianas que representan su posición real. Las aristas tienen costos asociados, como distancia, tiempo, límites de velocidad o incluso características como el número de cafeterías en el camino.
- **Grafos topológicos:** No requieren coordenadas cartesianas exactas. Por ejemplo, un robot puede reconocer un lugar como "cocina" y saber que atravesar una puerta lo lleva a la "sala de estar". Este enfoque es útil cuando no se dispone de información precisa sobre la posición.

La navegación basada en mapas combina modelos matemáticos como grafos y cuadrículas, las cuales se revisarán a detalle en el segundo informe del capítulo, para permitir que los robots planifiquen rutas óptimas considerando múltiples restricciones y factores del entorno. Esta capacidad es esencial para lograr un comportamiento autónomo eficiente,

aunque se enfrentan desafíos como la representación precisa del entorno y la complejidad de los cálculos en áreas grandes o con múltiples restricciones.

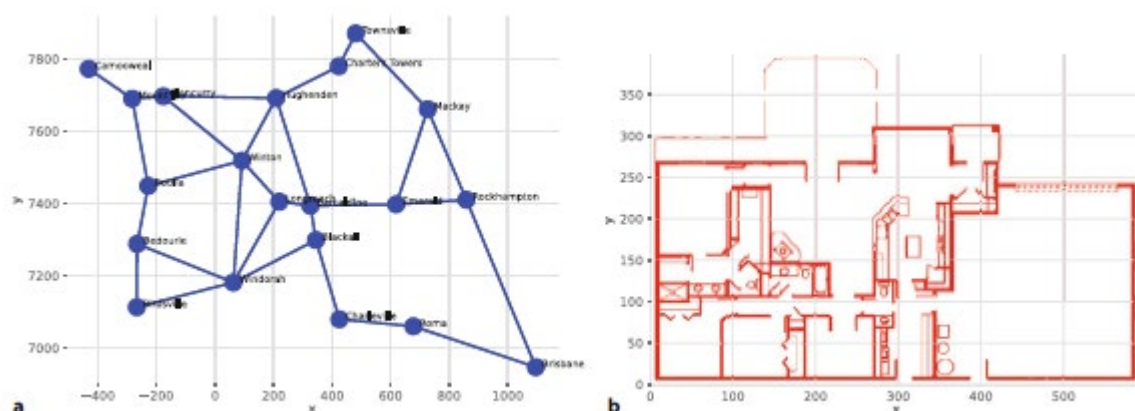


Figura 3. Representaciones comunes en navegación basada en mapas

5.3 Planeando recorridos con mapas basados en grafos matemáticos.

En esta sección se aborda el **planeamiento utilizando mapas basados en grafos**, un enfoque que utiliza una representación de un entorno como un grafo para facilitar la planificación de rutas. Los mapas basados en grafos son herramientas poderosas para la navegación de robots, ya que permiten representar de manera eficiente las relaciones entre los diferentes puntos del entorno y calcular la ruta óptima hacia un objetivo.

Representación de Grafos

En este tipo de planificación, los **vértices** del grafo representan ubicaciones en el mundo, y las **aristas** representan las conexiones entre estas ubicaciones, generalmente con un costo asociado como la distancia o el tiempo de viaje. Esta representación es muy útil para resolver el problema de planificación de rutas, ya que permite utilizar algoritmos de búsqueda para encontrar el camino más corto o más eficiente entre dos puntos.

El siguiente código carga un modelo de la casa y genera un mapa basado en el grafo de las ciudades, utilizando el formato **JSON** para describir la información de las ciudades y sus conexiones.

```
data = rtb_load_jsonfile("data/queensland.json");
✓ 0.0s

for name, info in data["places"].items():
    plot_point(info["utm"], text=name)
✓ 0.2s
```

Figura 3.1 Mapas basados en grafos matemáticos

Este código extrae los lugares de un archivo JSON, que contiene las ubicaciones de las ciudades en coordenadas **UTM** (Universal Transverse Mercator). Después, utiliza la librería **rtb** para representar estos lugares en un mapa gráfico, utilizando las coordenadas de cada ciudad.

Representación del Grafo y Ejecución del Algoritmo

Para continuar con el análisis del grafo, se utiliza el paquete **Python graph** para crear un objeto grafo, agregarle los vértices y aristas, y finalmente visualizarlo.

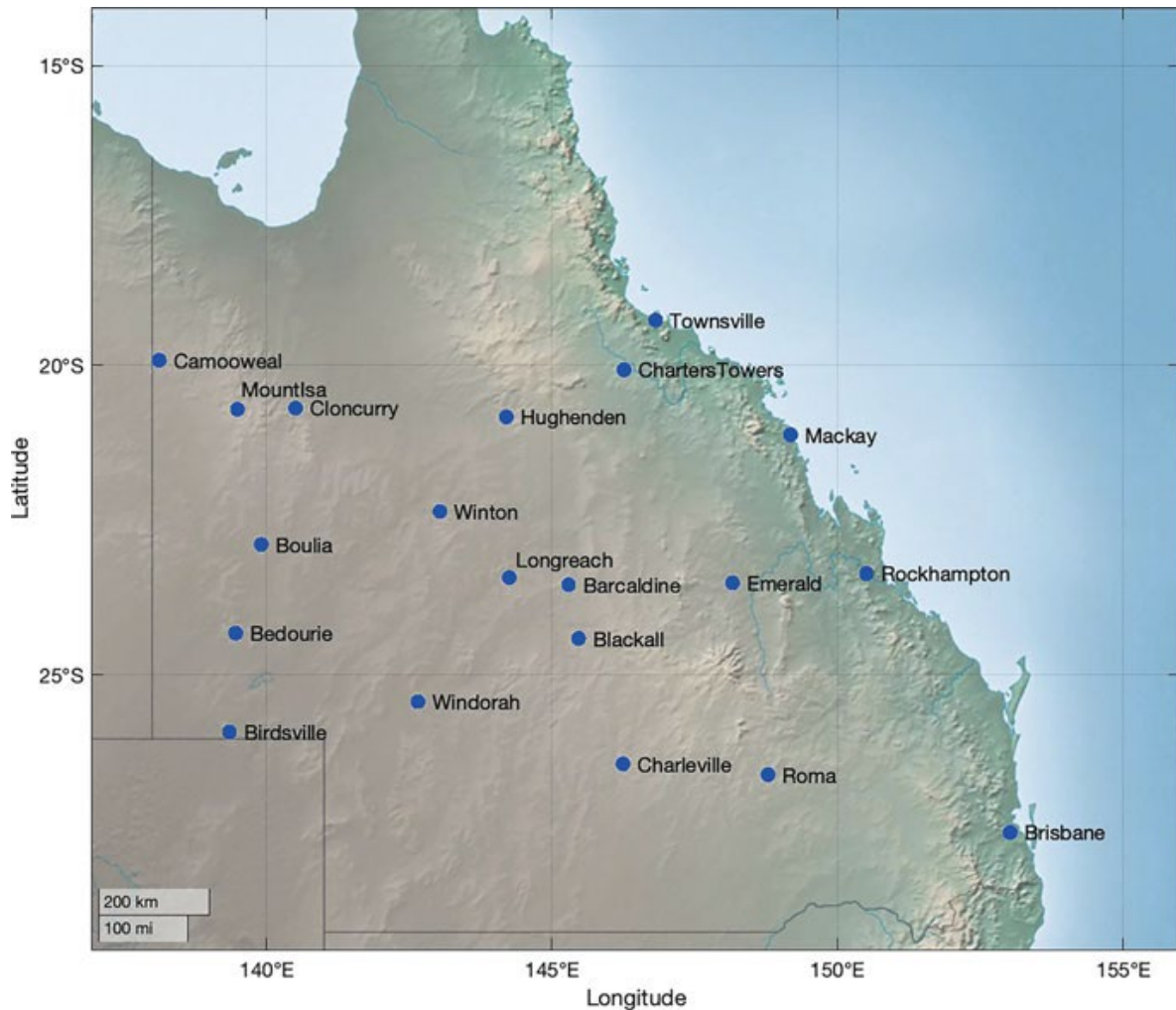


Figura 3.3 Mapas de las ciudades de Queensland

El objeto grafo tiene varios métodos, entre ellos `g.plot()`, que genera una visualización del grafo en 2D. La **Figura 3.3** muestra un mapa de las ciudades de Queensland, donde las ciudades se representan como puntos y las conexiones entre ellas como aristas. La visualización se realiza en una escala aproximada, con coordenadas de latitud y longitud representadas en un sistema de referencia global.

Algoritmos de Búsqueda en Grafos

Una vez que se ha representado el entorno en forma de grafo, se pueden aplicar algoritmos de búsqueda para encontrar la ruta más eficiente desde un punto de inicio hasta un objetivo.

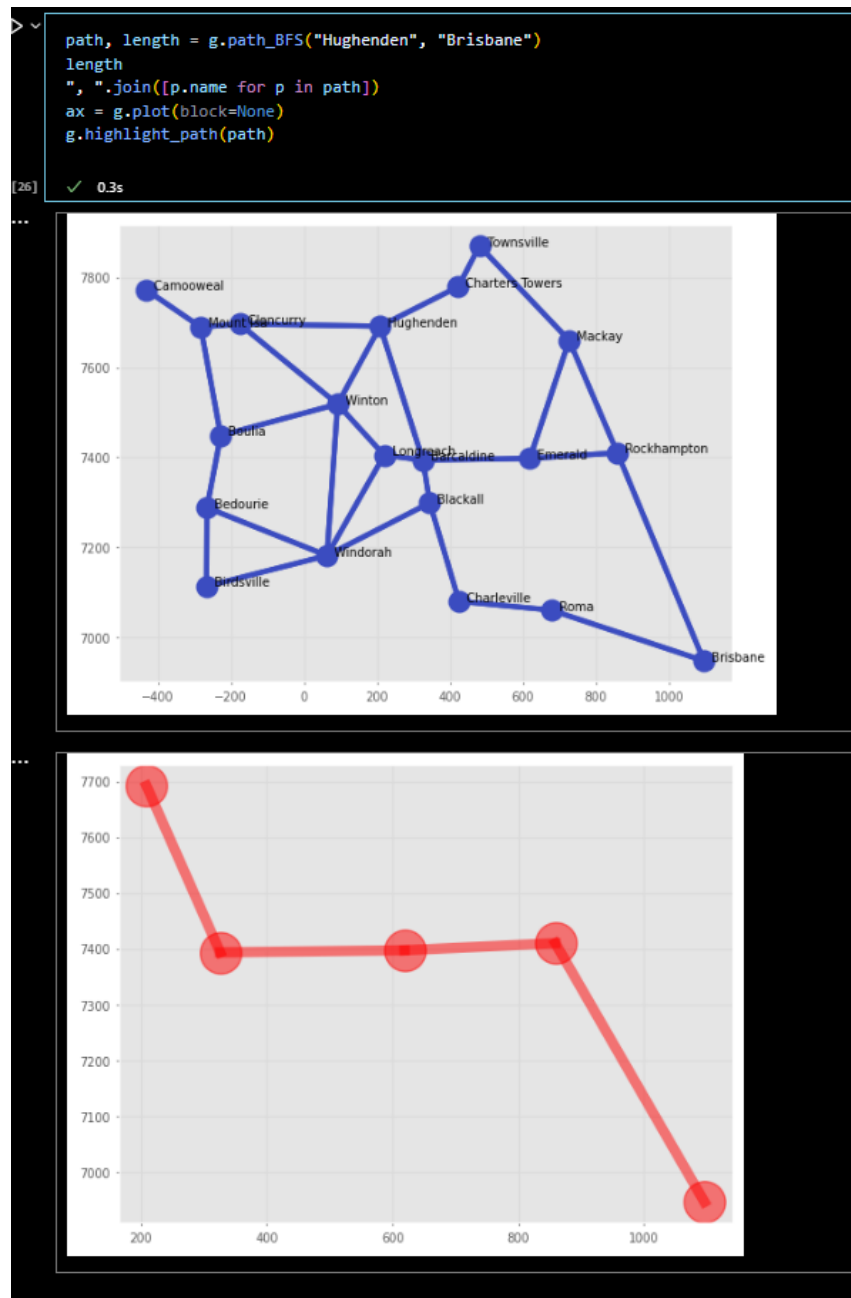


Figura 3.4 Traza de objetivos en el mapa de grafos.

1. Búsqueda en Amplitud (BFS):

Este algoritmo recorre todos los vértices del grafo en un orden de nivel por nivel, buscando el camino más corto en términos de número de aristas. En el ejemplo, el algoritmo encuentra una ruta con una longitud de **1759 km**, tal como se mira el recorrido en la figura 3.4.

2. **Búsqueda de Costo Uniforme (UCS):**

A diferencia de BFS, el algoritmo UCS considera el costo de las aristas y busca el camino de menor costo total. En el ejemplo, el algoritmo UCS encontró un camino más corto de **169 km** comparado con BFS, como se muestra en la **Figura 3.5**.

3. **Búsqueda A* (A-star):**

A* es un algoritmo de búsqueda que utiliza tanto el costo acumulado desde el inicio hasta el vértice actual, como una estimación del costo restante hasta el objetivo.

Este enfoque permite encontrar el camino óptimo en situaciones más complejas y es una extensión de UCS que utiliza una heurística. El algoritmo A* ha encontrado una ruta óptima más corta, lo que se visualiza en la **Figura 3.5**

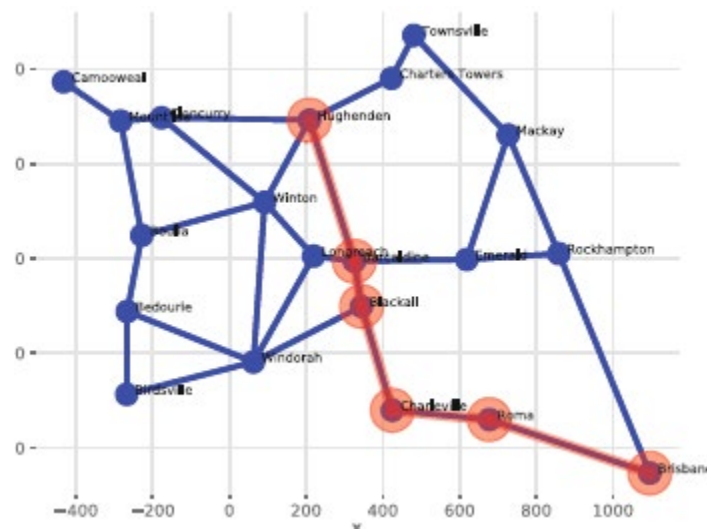


Figura 3.5 Mínimo tiempo de recorrido FCS + A*.

En esta sección, se aborda la **complejidad** y las propiedades de los algoritmos de planificación basados en grafos. Un grafo completo es aquel en el que se puede encontrar un camino entre cualquier par de vértices, mientras que los grafos disjuntos no tienen conexión entre todos sus vértices. La **complejidad** de un planificador está determinada por la cantidad de recursos computacionales y de memoria que requiere para encontrar una solución. Un planificador es **óptimo** si encuentra la mejor solución posible, considerando todos los costos involucrados en el camino. Sin embargo, un grafo completo y un planificador ideal pueden ser costosos en términos de memoria y tiempo de cómputo, lo que lleva a la necesidad de encontrar métodos más eficientes para la planificación en entornos complejos.

Conclusiones

- Las simulaciones demostraron que los robots con **navegación reactiva**, como los **vehículos Braitenberg**, logran alcanzar objetivos sin necesidad de planificación o mapas. Estos robots ajustan su movimiento de manera continua según los estímulos sensoriales, demostrando eficiencia en entornos dinámicos, aunque sin optimizar la ruta
- El control reactivo, basado en las diferencias entre los sensores izquierdo y derecho, permitió que el robot se moviera hacia un objetivo. Las gráficas de velocidad y tasa de giro confirmaron que el robot ajusta su dirección y velocidad según el entorno.
- Los algoritmos **BFS**, **UCS** y **A*** mostraron cómo un robot puede navegar eficientemente usando mapas basados en grafos. **A*** resultó más eficiente al integrar una heurística, optimizando la exploración del grafo, además, las simulaciones revelaron que **A*** es más eficiente que **BFS** y **UCS** en encontrar rutas óptimas, gracias a su heurística que reduce el número de nodos explorados.
- Las gráficas de las trayectorias del robot y las búsquedas en el grafo validaron la efectividad de los algoritmos, mostrando cómo los robots navegan en entornos complejos.

Tanto la **navegación reactiva** como la **planificación basada en grafos** son complementarias. La primera es útil en entornos dinámicos y la segunda en entornos más controlados. Ambas estrategias, junto con **ROS2** y sensores avanzados, forman la base para sistemas de navegación autónoma robustos.

1. Referencias

- UNTRACE folder.
- Corke, P. Robotics, Vision and Control v2 (RVC)
- notebook : chap5.ipynb
- notebook : chap5CP.ipynb
- Documentación Robotics Toolbox
- Git hub RVC v2